ID: C233106

Problem 1: Count Significant Digits

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    string num;
    cin >> num;

    int count = 0;
    bool start = false;

    for (char c : num) {
        if (c == '.')
            continue;
        if (c == '0' && !start)
            continue;

        start = true;
        count++;
    }

    cout << count << endl;

    return 0;
}
```

Problem 2: Round Half to Even (Banker's Rounding)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    double num;
    int n;
    cin >> num >> n;

    double p = pow(10, n);
    num = num * p;

    int x = (int)num;
    double r = num - x;

    if (r > 0.5)
        x++;
    else if (r == 0.5 && x % 2 != 0)
        x++;

    num = x / p;

    cout << fixed;
    cout.precision(n);
    cout << num << endl;

    return 0;
}
```

3. Evaluate Polynomial using Horner's Rule

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, x, i;
    cin >> n;

    int poly[n + 1];

    for (i = 0; i <= n; i++) {
        cin >> poly[i];
    }

    cin >> x;

    int result = poly[0];
    for (i = 1; i <= n; i++) {
        result = result * x + poly[i];
    }

    cout << result << endl;

    return 0;
}
```

4. Bisection Method

```
#include <bits/stdc++.h>
using namespace std;

double f(double x) {
    return x * x * x - x - 2;
}

int main() {
    double a, b, c;
    cin >> a >> b;

    while ((b - a) > 0.0001) {
        c = (a + b) / 2;
        if (f(a) * f(c) < 0)
            b = c;
        else
            a = c;
    }

    cout << (a + b) / 2 << endl;

    return 0;
}
```

5. Find All Roots using Bisection Method

```
#include <bits/stdc++.h>
using namespace std;

double f(double x) {
    return x * x * x - 6 * x + 4;
}

double bisection(double a, double b) {
    double c;
    while ((b - a) > 0.0005) {
        c = (a + b) / 2;
        if (f(a) * f(c) < 0)
            b = c;
        else
            a = c;
    }
    return (a + b) / 2;
}

int main() {
    cout << "Root 1 = " << bisection(-3, -2) << endl;
    cout << "Root 2 = " << bisection(0, 1) << endl;
    cout << "Root 3 = " << bisection(2, 3) << endl;

    return 0;
}
```

6. Newton-Raphson Method

```
#include <bits/stdc++.h>
using namespace std;

double f(double x) {
    return x * x * x - 6 * x + 4;
}

double df(double x) {
    return 3 * x * x - 6;
}

int main() {
    double x0 = 0.8;
    double x1;

    while (true) {
        x1 = x0 - f(x0) / df(x0);

        if (fabs(x1 - x0) < 0.0005)
            break;

        x0 = x1;
    }

    cout << fixed << setprecision(3) << x1 << endl;

    return 0;
}
```

7. False Position Method

```
#include <bits/stdc++.h>
using namespace std;

double f(double x) {
    return x * x * x - x + 2;
}

int main() {
    double a, b, c;
    cin >> a >> b;

    while (true) {
        c = (a * f(b) - b * f(a)) / (f(b) - f(a));

        if (fabs(f(c)) < 0.0005)
            break;

        if (f(a) * f(c) < 0)
            b = c;
        else
            a = c;
    }

    cout << fixed << setprecision(3) << c << endl;

    return 0;
}
```

8. Secant Method

```
#include <bits/stdc++.h>
using namespace std;

double f(double x) {
    return x * x * x - 5 * x * x - 29;
}

int main() {
    double x0, x1, x2;
    cin >> x0 >> x1;

    while (true) {
        x2 = x1 - (f(x1) * (x1 - x0)) / (f(x1) - f(x0));

        if (fabs(x2 - x1) < 0.0005)
            break;

        x0 = x1;
        x1 = x2;
    }

    cout << fixed << setprecision(3) << x2 << endl;

    return 0;
}
```

9. Synthetic Division / Quotient Polynomial

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    double a[] = {1, -5, 10, -8};
    double root = 2;
    double b[4];

    b[0] = a[0];
    for (int i = 1; i < 4; i++) {
        b[i] = a[i] + root * b[i - 1];
    }

    cout << "Quotient polynomial = ";
    cout << b[0] << "x^2 ";

    if (b[1] >= 0)
        cout << "+ " << b[1] << "x ";
    else
        cout << "- " << -b[1] << "x ";

    if (b[2] >= 0)
        cout << "+ " << b[2];
    else
        cout << "- " << -b[2];

    cout << endl;

    return 0;
}
```

10. Newton-Raphson with Deflation

```
#include <bits/stdc++.h>
using namespace std;

double f(double x) {
    return x * x * x - 6 * x + 4;
}

double df(double x) {
    return 3 * x * x - 6;
}

int main() {
    double x = 2.5;
    double x1;

    do {
        x1 = x - f(x) / df(x);
        x = x1;
    } while (fabs(f(x)) > 0.0005);

    double r1 = x;
    cout << "Root 1 = " << r1 << endl;

    double b1 = 1;
    double b2 = r1;
    double b3 = r1 * r1 - 6;

    double D = b2 * b2 - 4 * b1 * b3;
    double r2 = (-b2 + sqrt(D)) / (2 * b1);
    double r3 = (-b2 - sqrt(D)) / (2 * b1);

    cout << "Root 2 = " << r2 << endl;
    cout << "Root 3 = " << r3 << endl;

    return 0;
}
```